



F13 TECHNOLOGIES

INTERNSHIP PROJECT REPORT

Serverless Leave Management System Using Amazon Web Services (AWS)



SUBMITTED BY
TEAM NIKITA

Name	Role
• NIKITHA J	(Team Lead)
• ABDULLAH NORE	(Member)
• DIVYA PATEL	(Member)
• NISHANT NARUDKAR	(Member)

UNDER THE GUIDANCE OF
MR. KABIR RAO
MENTOR

YEAR

2026-27



ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **F13 Technologies** for providing us with the opportunity to undertake our internship and contribute to the development of the "**Serverless Leave Management System using Amazon Web Services (AWS)**." This internship provided us with valuable practical exposure to cloud computing, serverless application development, and modern software engineering practices.

We would like to express our heartfelt gratitude to **Mr. Kabir Rao**, our mentor at **F13 Technologies**, for his continuous guidance, encouragement, technical expertise, and invaluable support throughout the internship. His mentorship, valuable suggestions, and constructive feedback played a significant role in the successful completion of this project.

We would also like to thank the entire team at **F13 Technologies** for their cooperation, knowledge sharing, and continuous support. Working collaboratively with experienced professionals provided us with valuable insights into real-world software development, cloud technologies, teamwork, and industry best practices.

Abstract

1.1 Project Overview

Organizations often rely on manual leave management processes, including emails, spreadsheets, or paper forms, to track employee leave requests. These methods can lead to delays in approvals, inaccurate leave balance calculations, overlapping leave requests, and poor visibility for managers and HR departments.

The **Leave Management System** is a cloud-based web application developed using **Amazon Web Services (AWS)** to automate the complete leave approval process. Employees can submit leave requests through a web interface, managers can review and approve or reject requests, and HR can provide additional approval for long-duration leave requests. The application automatically validates leave balances, prevents overlapping leave applications, updates employee leave records, and sends email notifications at every stage of the approval workflow.

The project follows a **serverless architecture**, where AWS services such as Amazon API Gateway, AWS Lambda, Amazon DynamoDB, Amazon SES, Amazon S3, Amazon Cognito, and AWS Step

Functions work together to provide a scalable, secure, and cost-effective solution without the need to manage physical servers.

1.2 Background

Employee leave management is an essential function in every organization. As organizations grow, managing employee leave manually becomes increasingly difficult. Manual systems often result in calculation errors, approval delays, data inconsistency, and poor record management.

Cloud computing provides an efficient alternative by enabling organizations to automate business processes while reducing infrastructure costs. AWS offers a wide range of serverless services that simplify application development by eliminating server management responsibilities.

This project demonstrates how AWS serverless technologies can be integrated to build a modern Leave Management System capable of handling employee leave requests, approval workflows, database management, and email notifications efficiently.

1.3 Problem Statement

Many small and medium-sized organizations continue to manage employee leave using spreadsheets, emails, or paper-based forms. These approaches introduce several operational challenges, including:

- Manual leave approval process.
- Incorrect leave balance calculations.
- Difficulty tracking leave history.
- Lack of transparency for employees.
- Duplicate or overlapping leave requests.
- Delayed communication between employees, managers, and HR.
- Increased administrative workload for HR departments.

To address these issues, an automated leave management system is required that validates leave requests, enforces organizational policies, maintains leave balances automatically, and provides a structured approval workflow.

1.4 Existing System

In a traditional leave management process, employees submit leave requests through email or paper forms. Managers manually verify leave balances before approving or rejecting requests. HR departments maintain leave records using spreadsheets or standalone software, making it difficult to synchronize employee information across departments.

Since leave balances are updated manually, calculation mistakes and duplicate entries frequently occur. Employees also have limited visibility into the status of their leave requests, often requiring follow-up communication with managers or HR.

1.4.1 Limitations of Existing System

The existing manual approach has several limitations:

- Time-consuming approval process.
- Manual calculation of leave balances.
- High probability of human errors.
- No automatic validation of leave policies.
- Difficulty detecting overlapping leave requests.
- Lack of centralized data management.
- Limited reporting capabilities.
- Poor scalability for large organizations.
- Delayed employee notifications.
- Increased administrative effort.

1.5 Proposed System

The proposed Leave Management System automates the complete leave approval workflow using AWS serverless services. Employees can apply for leave through a web application, managers review the request, and HR performs additional approval when required. The system validates leave balances, prevents duplicate leave requests, maintains leave records in DynamoDB, and sends email notifications using Amazon SES.

All business logic is implemented using AWS Lambda functions, while API Gateway exposes REST APIs for communication between the frontend and backend. Amazon Cognito manages user authentication, and Amazon S3 hosts the static web application.

1.5.1 Advantages of Proposed System

The proposed system offers several benefits:

- Fully automated leave application process.
- Real-time leave balance validation.
- Prevention of overlapping leave requests.
- Configurable leave policies.
- Faster approval workflow.
- Automatic email notifications.
- Secure user authentication.
- Serverless architecture with reduced operational cost.
- Highly scalable and reliable.
- Easy integration with other AWS services.

1.6 Objectives

The primary objectives of this project are:

- Design and develop a cloud-based Leave Management System.
- Automate employee leave application and approval.
- Validate leave balances before submission.
- Detect overlapping leave requests.
- Maintain accurate leave records.
- Implement manager and HR approval workflows.
- Provide secure user authentication using Amazon Cognito.
- Send automated email notifications using Amazon SES.
- Demonstrate serverless application development using AWS.

1.7 Scope of the Project

The project focuses on automating leave management within an organization using AWS cloud services. The application supports multiple user roles, including employees, managers, and HR administrators. It enables employees to submit leave requests, managers to approve or reject requests, and HR to handle long-duration leave approvals.

The system maintains leave balances, validates business rules, stores data in DynamoDB, and provides a user-friendly web interface. While the application demonstrates a complete leave management workflow, advanced enterprise features such as payroll integration, biometric attendance synchronization, and mobile applications are considered future enhancements.

1.8 Project Features

The Leave Management System includes the following features:

- Employee login using Amazon Cognito.
- Leave application through a web interface.
- Support for multiple leave types.
- Automatic leave balance validation.
- Overlapping leave detection.
- Manager approval workflow.
- HR approval for long-duration leave requests.
- Dynamic leave policy configuration.
- Real-time leave status tracking.
- Leave history dashboard.
- Automatic email notifications using Amazon SES.
- RESTful API architecture using API Gateway.
- Serverless backend using AWS Lambda.
- Data storage using Amazon DynamoDB.
- Static website hosting using Amazon S3.
- Scalable and secure cloud architecture.

Chapter 2 – Literature Survey

2.1 Cloud Computing

Cloud computing is a technology that enables users to access computing resources such as servers, storage, databases, networking, and software over the internet instead of relying on local infrastructure. It provides organizations with the flexibility to deploy applications quickly while reducing hardware and maintenance costs.

Cloud service providers offer resources on a pay-as-you-go basis, allowing businesses to scale applications according to demand without investing in physical infrastructure. Cloud computing is

generally categorized into three service models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS).

In this project, cloud computing forms the foundation of the Leave Management System by hosting the application entirely on Amazon Web Services (AWS). The use of cloud services ensures high availability, scalability, security, and cost efficiency while eliminating the need for dedicated server management.

2.2 AWS Serverless Computing

Serverless computing is a cloud execution model in which the cloud provider automatically manages the underlying infrastructure, including servers, operating systems, and resource scaling. Developers focus solely on writing business logic while the cloud platform handles deployment, execution, monitoring, and scaling.

Amazon Web Services provides several serverless services that simplify application development. AWS Lambda executes backend functions without requiring dedicated servers. Amazon API Gateway securely exposes REST APIs to frontend applications. Amazon DynamoDB provides a fully managed NoSQL database for storing application data, while Amazon SES enables email notifications. Amazon Cognito handles user authentication, and AWS Step Functions coordinate multi-step workflows.

The Leave Management System utilizes these services to create a completely serverless application capable of processing leave requests efficiently while minimizing infrastructure management.

2.3 Leave Management Systems

A Leave Management System is an application designed to automate the process of requesting, approving, tracking, and managing employee leave within an organization. Modern leave management systems replace traditional paper-based or spreadsheet-based methods with centralized digital platforms.

Typical functionalities include employee leave applications, leave balance management, approval workflows, leave history tracking, reporting, and automated notifications. Organizations often implement different leave policies for sick leave, casual leave, earned leave, maternity leave, and unpaid leave.

An efficient leave management system improves transparency, reduces administrative workload, minimizes errors, and enables employees to monitor the status of their requests in real time.

2.4 Existing Leave Management Solutions

Many commercial leave management solutions are available, including enterprise Human Resource Management Systems (HRMS) and cloud-based attendance platforms. These applications generally provide features such as employee leave tracking, automated approval workflows, reporting, payroll integration, and calendar synchronization.

However, many of these commercial systems require paid subscriptions, extensive configuration, or infrastructure that may not be suitable for small organizations or educational projects. Some organizations continue using spreadsheets or manual approval methods due to budget constraints or existing operational practices.

This project demonstrates how similar functionality can be achieved using AWS serverless technologies while maintaining flexibility and reducing infrastructure costs.

2.5 Comparative Analysis

The following table compares the traditional leave management approach with the proposed AWS-based serverless solution.

Feature	Traditional System	Proposed AWS System
Leave Application	Paper Forms / Email	Web Application
Approval Process	Manual	Automated Workflow
Leave Balance	Manual Calculation	Automatic Validation
Database	Spreadsheets	Amazon DynamoDB
Notifications	Manual Communication	Amazon SES
Authentication	Basic Login	Amazon Cognito

Scalability	Limited	High
Maintenance	High	Low
Infrastructure	Physical Servers	Serverless Cloud
Availability	Organization Dependent	AWS Managed Services
Security	Basic	IAM and Cognito
Cost Model	Fixed Infrastructure Cost	Pay-as-you-go

Chapter Summary

The literature survey demonstrates that cloud computing and serverless technologies provide an efficient platform for developing scalable enterprise applications. Existing leave management approaches often rely on manual processes or expensive enterprise software. By utilizing AWS serverless services such as Lambda, API Gateway, DynamoDB, SES, Cognito, and Step Functions, the proposed Leave Management System delivers an automated, secure, scalable, and cost-effective solution suitable for modern organizational requirements.

Chapter 3 – System Requirements

3.1 Functional Requirements

Functional requirements describe the core functionalities that the Leave Management System must perform in order to meet user and organizational requirements.

The proposed system consists of three primary user roles: Employee, Manager, and Human Resources (HR). Each user is provided with role-specific functionalities through a secure authentication mechanism.

The major functional requirements of the system are listed below.

Employee Module

The employee module provides the following functionalities:

- Secure login using Amazon Cognito.
- View available leave balance.
- Submit leave application.
- Select leave type (Sick Leave, Casual Leave, Earned Leave, Unpaid Leave).
- Enter leave start and end dates.
- Provide reason for leave.
- View leave request history.
- Track current leave status.
- Receive email notifications regarding leave approval or rejection.

Manager Module

The manager module allows managers to review employee leave requests assigned to their team.

The manager can:

- View pending leave requests.
- Review employee leave details.
- Approve leave requests.
- Reject leave requests.
- Add comments while approving or rejecting.
- Forward long-duration leave requests (>5 days) to HR for further approval.

Human Resources (HR) Module

The HR module provides administrative control over long-duration leave requests.

HR users can:

- View requests awaiting HR approval.
- Approve leave requests.
- Reject leave requests.
- Enter HR remarks.
- Finalize leave approval.
- Trigger employee notification after final decision.

Leave Validation Module

The application validates leave requests before storing them in the database.

The system performs the following validations:

- Checks whether the selected leave type exists.
- Verifies employee leave balance.
- Detects overlapping leave requests.
- Prevents duplicate leave applications.
- Ensures valid leave dates.
- Prevents invalid leave durations.

Notification Module

The system automatically generates email notifications during important workflow stages.

Notifications are sent when:

- Leave request is submitted.
- Manager approves leave.
- Manager rejects leave.
- HR approves leave.
- HR rejects leave.

Amazon SES is used to send all notification emails.

3.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes of the system.

Performance

The application should process leave requests with minimal response time. Since AWS Lambda scales automatically, the backend can handle multiple requests simultaneously without manual intervention.

Scalability

The application is designed using AWS serverless services that automatically scale based on incoming traffic. No additional server configuration is required.

Availability

AWS cloud services provide high availability by operating across multiple availability zones, ensuring that the application remains accessible with minimal downtime.

Reliability

The use of managed AWS services such as DynamoDB, Lambda, API Gateway, and SES improves overall application reliability by reducing infrastructure failures.

Security

Security is maintained through multiple AWS services.

The system provides:

- User authentication using Amazon Cognito.
- Secure API communication.
- IAM-based permissions.
- Protected backend services.
- Controlled database access.

Maintainability

The application follows a modular serverless architecture where each AWS Lambda function performs a specific responsibility. This simplifies debugging, testing, and future enhancements.

Cost Effectiveness

AWS serverless services follow a pay-as-you-use pricing model, allowing the organization to pay only for the resources consumed instead of maintaining dedicated infrastructure.

3.3 Hardware Requirements

Since the application is hosted on AWS Cloud, minimal client-side hardware is required.

Component	Specification
Processor	Intel Core i5 or higher
RAM	8 GB Minimum
Storage	256 GB SSD
Internet	Stable Broadband Connection

Client Machine

Any modern desktop or laptop capable of running a web browser can access the application.

Supported browsers include:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

3.4 Software Requirements

The following software components were used during development.

Software	Purpose
Windows 11	Operating System

Visual Studio Code	Source Code Editor
AWS Management Console	Cloud Resource Management
Python 3.x	Lambda Function Development
HTML5	Frontend Structure
CSS3	User Interface Styling
JavaScript	Frontend Logic
Amazon Cognito	Authentication
Amazon API Gateway	REST APIs
AWS Lambda	Backend Functions
Amazon DynamoDB	Database
Amazon SES	Email Notifications
AWS Step Functions	Workflow Management
Git & GitHub	Version Control

3.5 AWS Services Used

The Leave Management System is built entirely on AWS serverless technologies.

Amazon S3

Amazon S3 is used to host the frontend static website, including HTML, CSS, JavaScript, and image assets.

Amazon Cognito

Amazon Cognito provides secure user authentication and authorization. Users log in using Cognito before accessing their respective dashboards.

Amazon API Gateway

API Gateway exposes RESTful APIs that allow the frontend to communicate securely with backend Lambda functions.

AWS Lambda

AWS Lambda executes all backend business logic.

Different Lambda functions are responsible for:

- Leave Submission
- Manager Approval
- HR Approval
- Leave Balance Retrieval
- Pending Request Retrieval

Amazon DynamoDB

DynamoDB stores all application data.

The project uses three primary tables:

- leave_requests
- leave_balances
- leave_config

Amazon SES

Amazon Simple Email Service (SES) sends email notifications whenever the leave status changes.

AWS Step Functions

AWS Step Functions coordinate multi-step approval workflows, particularly for leave requests requiring additional HR approval.

AWS IAM

IAM controls access permissions between AWS services, ensuring that Lambda functions only access the resources required for their operation.

Chapter Summary

This chapter discussed the functional and non-functional requirements of the proposed Leave Management System. It also described the hardware and software environment required for development and deployment. Finally, it introduced the AWS services that form the foundation of the application's serverless architecture, demonstrating how each service contributes to the overall functionality of the system.

Chapter 4 – System Architecture

4.1 Overall Architecture

The Leave Management System follows a **serverless multi-tier architecture** built entirely on Amazon Web Services (AWS). The application is divided into three major layers: the Presentation Layer (Frontend), the Application Layer (Backend), and the Data Layer (Database). Each layer communicates securely using AWS managed services, ensuring scalability, reliability, and minimal infrastructure management.

The frontend of the application is hosted as a static website on Amazon S3. Users authenticate through Amazon Cognito before accessing the application. Once authenticated, API requests are sent to Amazon API Gateway, which acts as the entry point for all backend operations. API Gateway invokes the appropriate AWS Lambda functions that implement the business logic. The Lambda functions interact with Amazon DynamoDB to store and retrieve application data. Amazon SES is responsible for sending automated email notifications whenever the leave request status changes.

For leave requests requiring multiple approval levels, AWS Step Functions coordinate the workflow between the manager and HR approval process.

This architecture eliminates the need for traditional application servers while providing automatic scaling, fault tolerance, and simplified deployment.

4.2 AWS Architecture Diagram

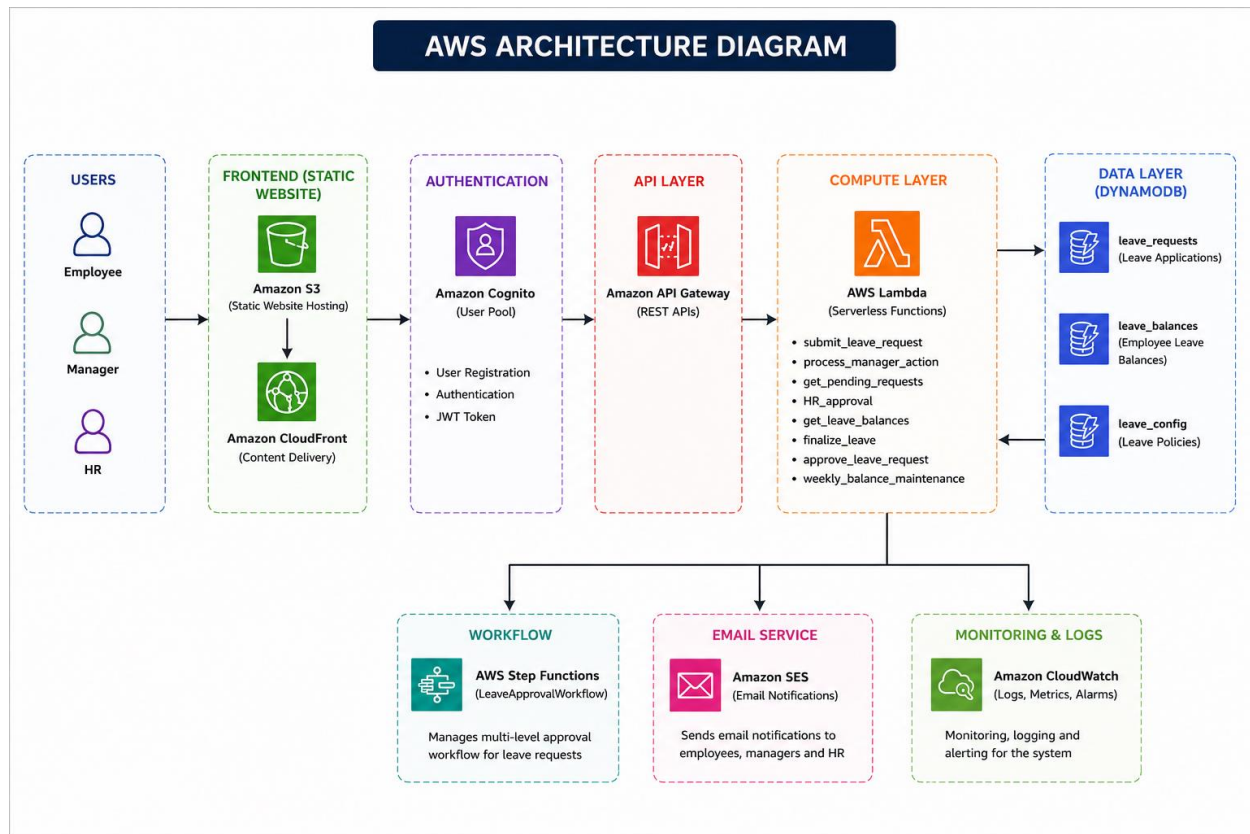


Figure 4.1: AWS Architecture of the Leave Management System

The overall AWS architecture consists of the following components:

- Amazon S3 hosts the static frontend files including HTML, CSS, and JavaScript.
- Amazon Cognito authenticates employees, managers, and HR users.
- Amazon API Gateway exposes REST APIs for frontend communication.
- AWS Lambda executes backend business logic.
- Amazon DynamoDB stores leave requests, employee leave balances, and leave configuration data.
- Amazon SES sends email notifications during various stages of leave processing.
- AWS Step Functions coordinate the approval workflow for leave requests requiring HR approval.

The architecture provides a highly scalable and secure solution while minimizing operational overhead through serverless computing.

4.3 Frontend Architecture

The frontend of the Leave Management System provides role-based user interfaces for Employees, Managers, and HR personnel. It is developed using HTML, CSS, and JavaScript and hosted as a static website on Amazon S3.

The frontend consists of the following modules:

Login Module

Responsible for authenticating users using Amazon Cognito.

Features include:

- User Login
- Authentication Token Generation
- Session Management
- Role Identification

Employee Dashboard

The Employee Dashboard allows employees to perform all leave-related activities.

Features include:

- View Leave Balance
- Apply for Leave
- View Leave History
- Track Leave Status
- Receive Status Notifications

Manager Dashboard

Managers receive pending leave requests submitted by employees.

The dashboard provides:

- View Pending Requests
- Approve Leave
- Reject Leave

- Add Manager Comments
- Forward Long Leave Requests to HR

HR Dashboard

The HR Dashboard is responsible for processing long-duration leave requests.

HR users can:

- View Pending HR Requests
- Approve Requests
- Reject Requests
- Add HR Remarks
- Finalize Leave Decision

4.4 Backend Architecture

The backend of the Leave Management System follows a serverless event-driven architecture using AWS Lambda functions. Each Lambda function performs a specific business operation.

The primary Lambda functions are:

Submit Leave Lambda

Responsibilities:

- Receive leave application.
- Validate leave type.
- Check leave balance.
- Detect overlapping leave requests.
- Store leave request.
- Send submission email.

Manager Approval Lambda

Responsibilities:

- Retrieve pending request.
- Validate current request status.

- Approve or reject leave.
- Update leave status.
- Forward requests exceeding five days to HR.
- Send manager decision email.

HR Approval Lambda

Responsibilities:

- Process leave requests awaiting HR approval.
- Finalize leave status.
- Deduct leave balance after approval.
- Store HR comments.
- Send final email notification.

Leave Balance Lambda

Responsibilities:

- Retrieve employee leave balance.
- Calculate remaining leave.
- Return balance details to frontend.

Pending Requests Lambda

Responsibilities:

- Retrieve pending leave requests for managers.
- Retrieve pending HR requests.
- Return data through API Gateway.

4.5 Database Architecture

The application stores all business data using Amazon DynamoDB.

Three tables are used in the system.

leave_requests

Stores all leave applications submitted by employees.

Key attributes include:

- employee_id
- request_id
- leave_type
- start_date
- end_date
- num_days
- reason
- status
- manager_comments
- hr_comments
- employee_email
- submitted_at

leave_balances

Stores annual leave balances.

Attributes include:

- employee_id
- leave_type_year
- total
- used
- remaining

leave_config

Stores configurable leave policies.

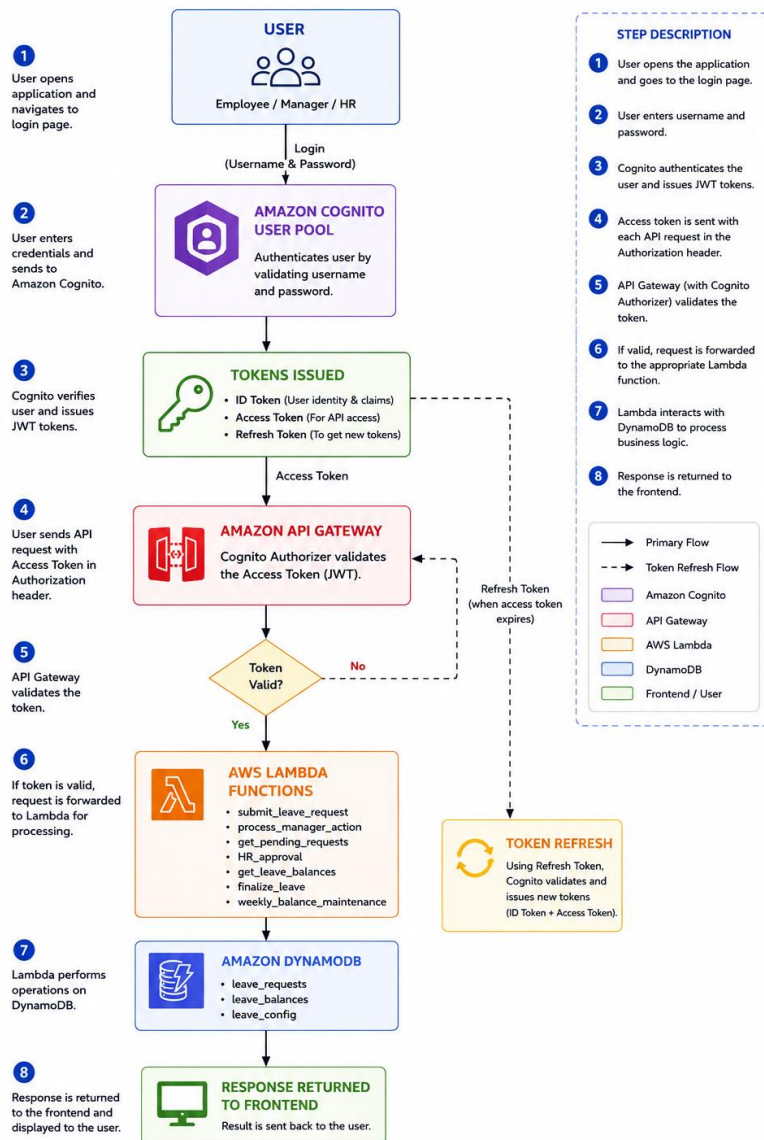
Attributes include:

- leave_type
- annual_quota
- carry_forward

- approval_required

This table allows HR to modify leave rules without changing application code.

4.6 Authentication Flow



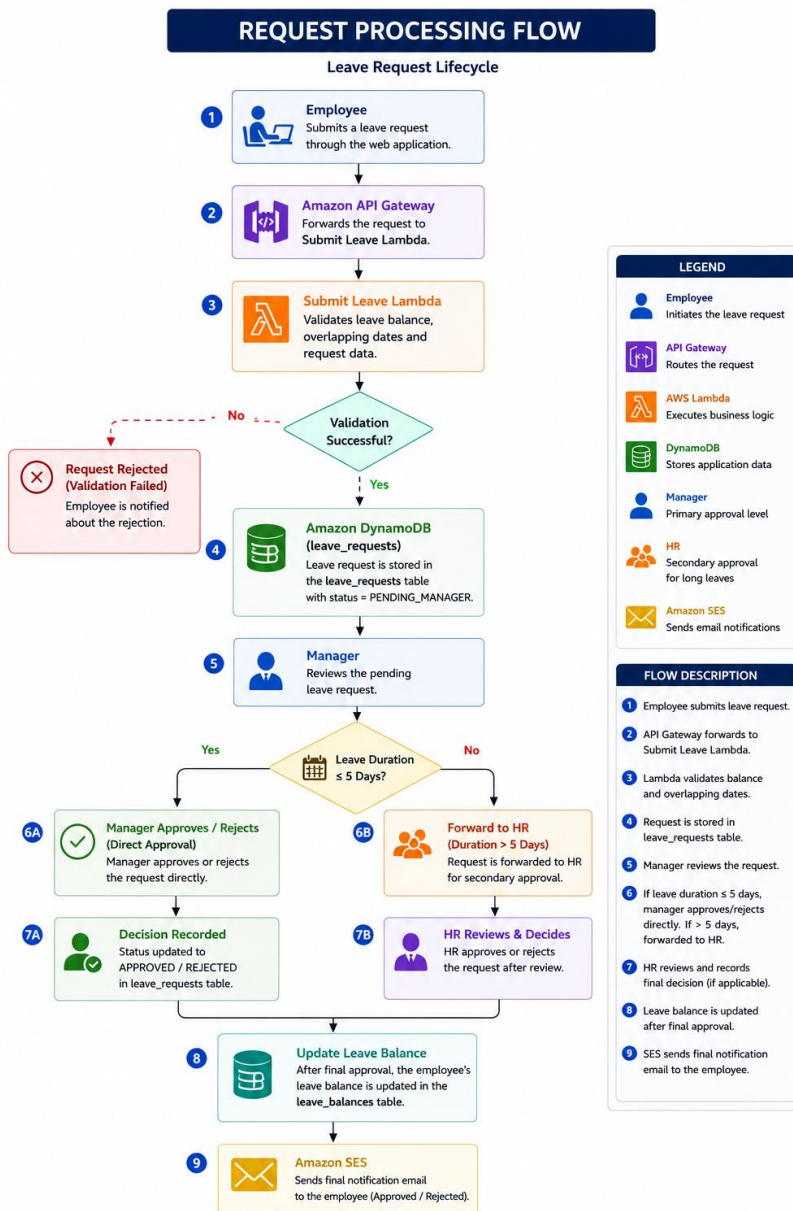
The authentication process follows these steps:

1. User accesses the web application hosted on Amazon S3.

2. Login credentials are verified by Amazon Cognito.
3. Cognito generates authentication tokens upon successful login.
4. The frontend stores the authentication token.
5. API requests include the authentication token.
6. API Gateway validates the token before invoking Lambda functions.
7. Authorized users gain access to role-specific dashboards.

This authentication mechanism ensures that only authenticated users can access protected application resources.

4.7 Request Processing Flow



The leave request lifecycle follows the sequence below:

1. Employee submits a leave request.
2. API Gateway forwards the request to the Submit Leave Lambda.
3. The Lambda validates leave balance and overlapping dates.
4. Leave request is stored in the `leave_requests` table.
5. Manager reviews the pending request.

6. If the leave duration is five days or less, the manager can approve or reject the request directly.
7. If the leave duration exceeds five days, the request is forwarded to HR for secondary approval.
8. HR reviews the request and records the final decision.
9. The leave balance is updated after final approval.
10. Amazon SES sends the final notification email to the employee.

Chapter Summary

This chapter explained the overall system architecture of the Leave Management System developed using AWS serverless services. It described the frontend, backend, database, authentication process, and request workflow in detail. The serverless architecture ensures scalability, security, cost efficiency, and simplified maintenance while providing a reliable platform for automating employee leave management.

Chapter 5 – Database Design

5.1 Introduction

A database plays a vital role in storing, retrieving, and managing application data efficiently. Since the Leave Management System is built using a serverless architecture, Amazon DynamoDB was selected as the primary database service. DynamoDB is a fully managed NoSQL database that provides high performance, automatic scaling, low latency, and seamless integration with other AWS services.

Unlike traditional relational databases, DynamoDB stores data as key-value pairs and documents, making it well suited for serverless applications where rapid data access and scalability are essential. The Leave Management System uses DynamoDB to maintain employee leave requests, annual leave balances, and leave policy configurations.

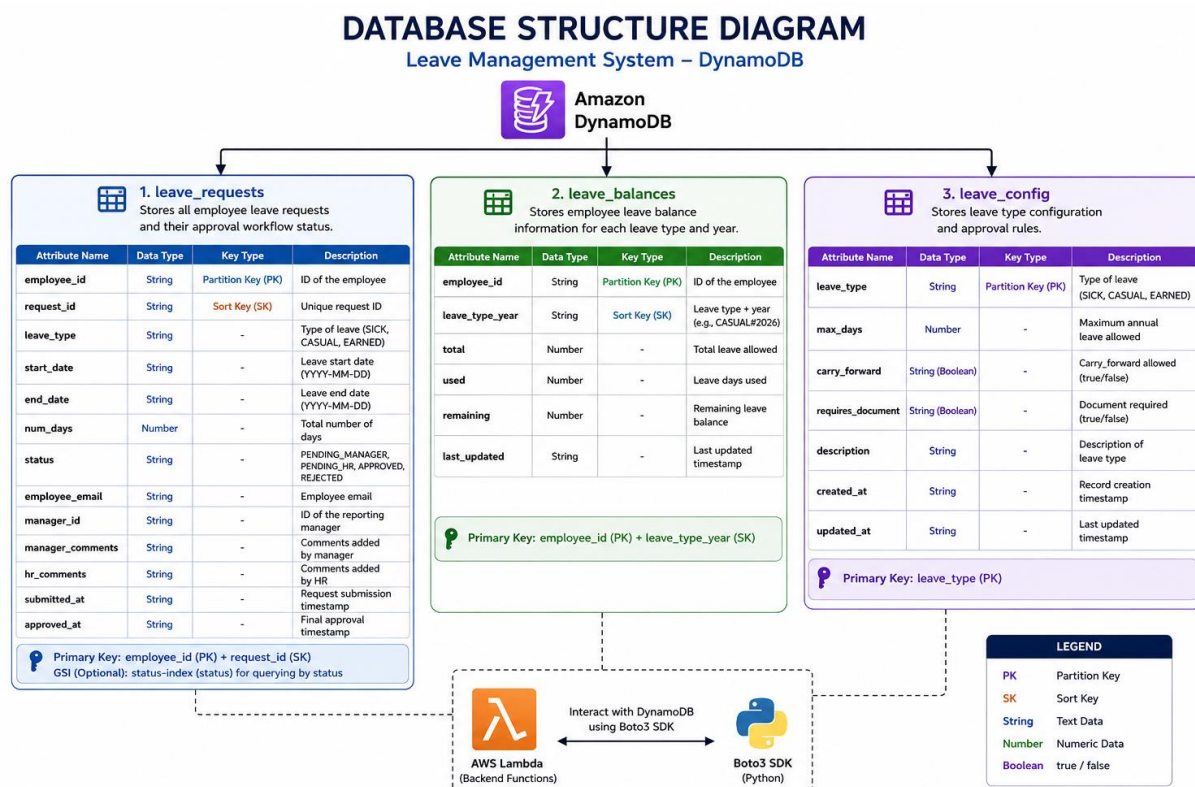
5.2 Database Overview

The application stores its information in three DynamoDB tables:

- **leave_requests**
- **leave_balances**
- **leave_config**

Each table has a specific responsibility, ensuring that data remains organized and easily accessible.

The following figure illustrates the database organization.



5.3 leave_requests Table

The **leave_requests** table stores every leave application submitted by employees. Each request is uniquely identified using the employee ID and request ID.

Primary Key

Attribute	Type
employee_id	Partition Key
request_id	Sort Key

Important Attributes

Attribute	Description
employee_id	Unique employee identifier
request_id	Unique leave request number
leave_type	Type of leave applied
start_date	Leave start date
end_date	Leave end date
num_days	Total leave duration
reason	Employee reason for leave
status	Current leave status
manager_comments	Manager remarks
hr_comments	HR remarks
employee_email	Employee email

submitted_at	Request submission timestamp
---------------------	------------------------------

Status Values

A leave request may have one of the following statuses:

- PENDING
- PENDING_HR
- APPROVED
- REJECTED

5.4 leave_balances Table

The **leave_balances** table maintains yearly leave balances for every employee.

Instead of calculating leave balances repeatedly, the application updates the remaining leave whenever a request receives final approval.

Primary Key

Attribute	Type
employee_id	Partition Key
leave_type_year	Sort Key

Important Attributes

Attribute	Description
total	Annual leave quota
used	Leave already utilized
remaining	Remaining leave balance

The leave balance is updated only after the leave request receives final approval.

5.5 leave_config Table

The **leave_config** table stores leave policies defined by HR.

Instead of hardcoding leave rules inside Lambda functions, the application retrieves leave configuration dynamically from this table

Primary Key

Attribute	Type
leave_type	Partition Key

Sample Attributes

Attribute	Description
leave_type	Leave category
annual_quota	Maximum annual leave
carry_forward	Carry-forward policy
approval_required	Additional approval requirement

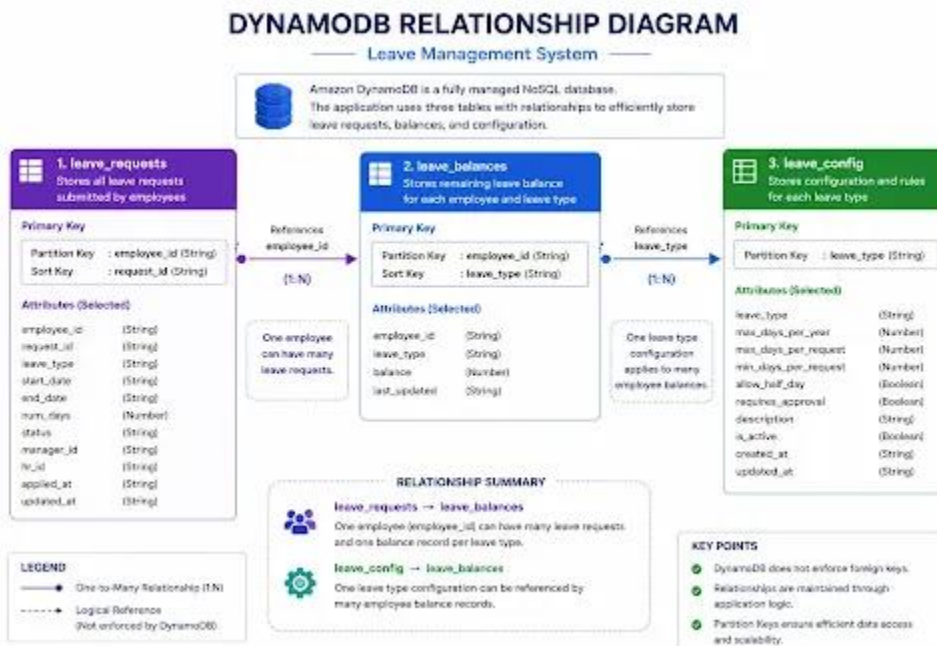
This approach allows HR to modify leave policies without changing application code.

5.6 Relationship Between Tables

Although DynamoDB is a NoSQL database and does not enforce relationships like relational databases, logical relationships exist between the three tables.



The system retrieves leave policies from **leave_config**, verifies employee balances from **leave_balances**, and finally stores leave requests in **leave_requests**.



(Insert Figure 5.2 – DynamoDB Relationship Diagram)

5.7 Data Flow Between Tables

Whenever an employee submits a leave request, the application performs several database operations.



5.8 Advantages of Using DynamoDB

Amazon DynamoDB provides several benefits for this project.

- Fully managed NoSQL database.
- Automatic scaling based on workload.
- High availability across AWS infrastructure.
- Low latency for read and write operations.
- Seamless integration with AWS Lambda.
- No server administration required.
- Cost-effective pay-as-you-use pricing.
- High durability and reliability.
- Suitable for serverless applications.

5.9 Database Security

The application secures database access using AWS Identity and Access Management (IAM).

Security measures include:

- IAM roles for Lambda functions.
- Restricted database permissions.
- No direct public access to DynamoDB.
- API Gateway acts as the secure entry point.
- User authentication through Amazon Cognito.
- Backend-only database operations.

These measures ensure that sensitive employee information remains protected.

Chapter Summary

This chapter described the database architecture of the Leave Management System. Three Amazon DynamoDB tables—**leave_requests**, **leave_balances**, and **leave_config**—are used to manage leave requests, employee leave balances, and leave policies. The chapter

also explained the logical relationship between these tables, the data flow during leave processing, and the advantages of using DynamoDB as a fully managed NoSQL database within a serverless AWS architecture.

Chapter 6 – System Implementation

6.1 Introduction

The implementation phase involves converting the proposed system design into a fully functional application. The Leave Management System was developed using a serverless architecture on Amazon Web Services (AWS), enabling scalability, security, and efficient resource utilization without managing physical servers.

The frontend was developed using HTML, CSS, and JavaScript and deployed on Amazon S3 as a static website. Backend functionality was implemented using AWS Lambda functions, with Amazon API Gateway exposing REST APIs. Amazon DynamoDB stores application data, Amazon Cognito manages authentication, AWS Step Functions coordinate approval workflows, and Amazon SES delivers email notifications.

The implementation focuses on modular development, allowing each AWS service to perform a dedicated responsibility while communicating through secure APIs.

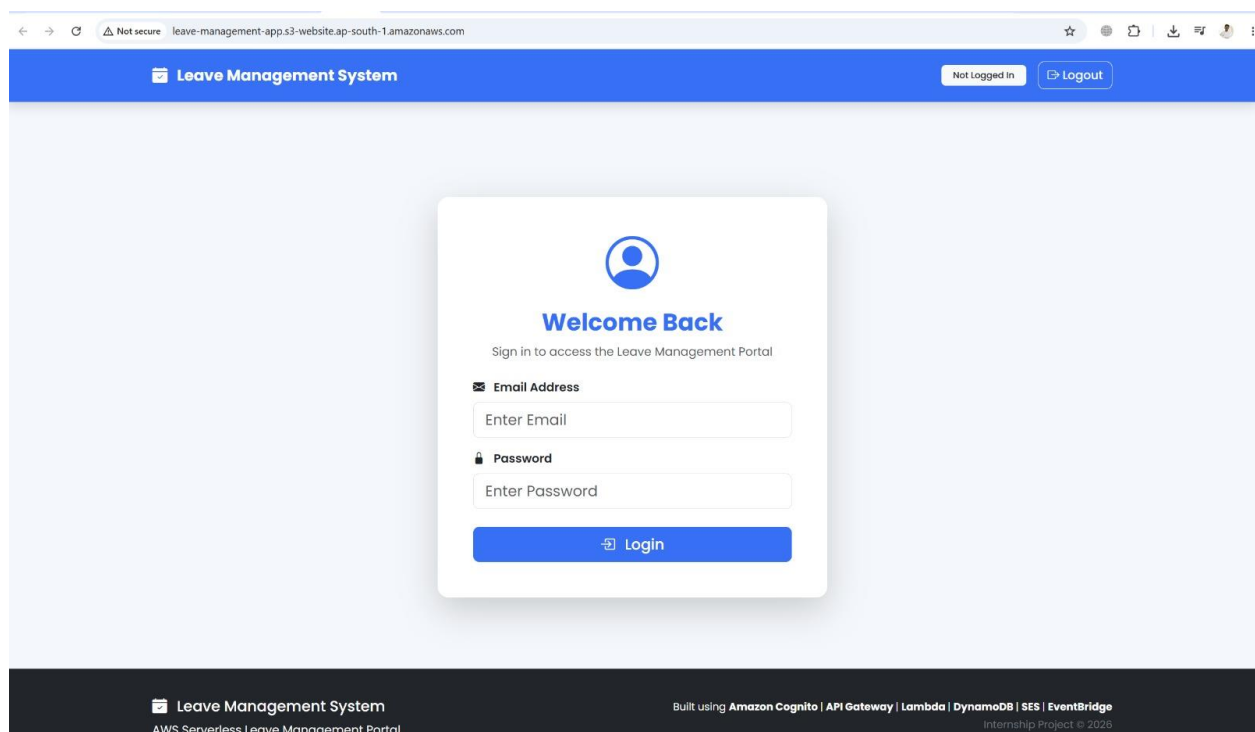
6.2 Frontend Implementation

The user interface was designed to provide a simple and intuitive experience for employees, managers, and HR personnel. HTML was used to structure the webpages, CSS provided responsive styling, and JavaScript handled user interactions and API communication.

The application consists of separate dashboards for Employees, Managers, and HR users. Each dashboard displays only the information relevant to the logged-in user's role. Users interact with the backend exclusively through REST APIs provided by Amazon API Gateway.

Major Frontend Modules

- Login Page
- Employee Dashboard
- Manager Dashboard
- HR Dashboard
- Leave Application Form
- Leave Balance Screen
- Pending Requests Screen
- Leave History Screen



6.3 Authentication Implementation

User authentication is implemented using Amazon Cognito. Each user must log in with valid credentials before accessing the application.

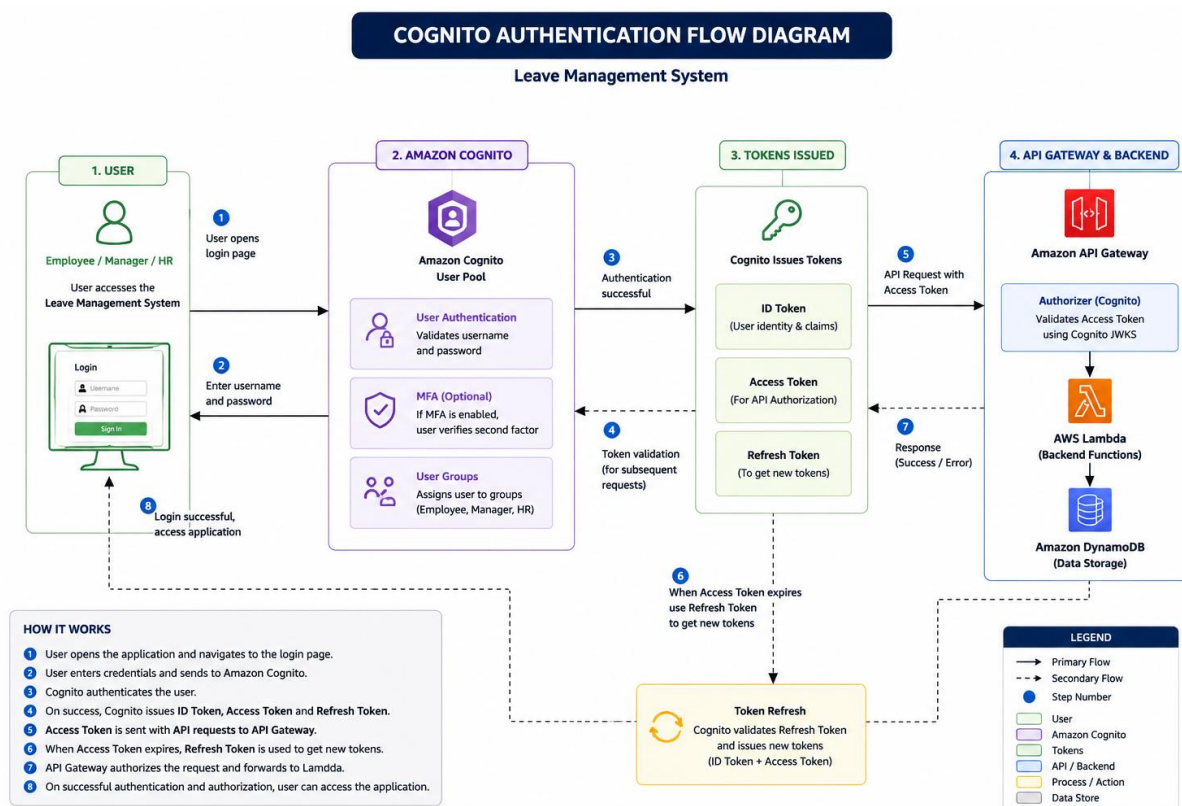
The authentication process includes:

- User Login

- Identity Verification
- Token Generation
- Token Validation
- Session Management

After successful authentication, Cognito issues secure JSON Web Tokens (JWT), which are attached to API requests. API Gateway validates these tokens before invoking backend Lambda functions.

This mechanism prevents unauthorized access to protected resources.



6.4 API Gateway Implementation

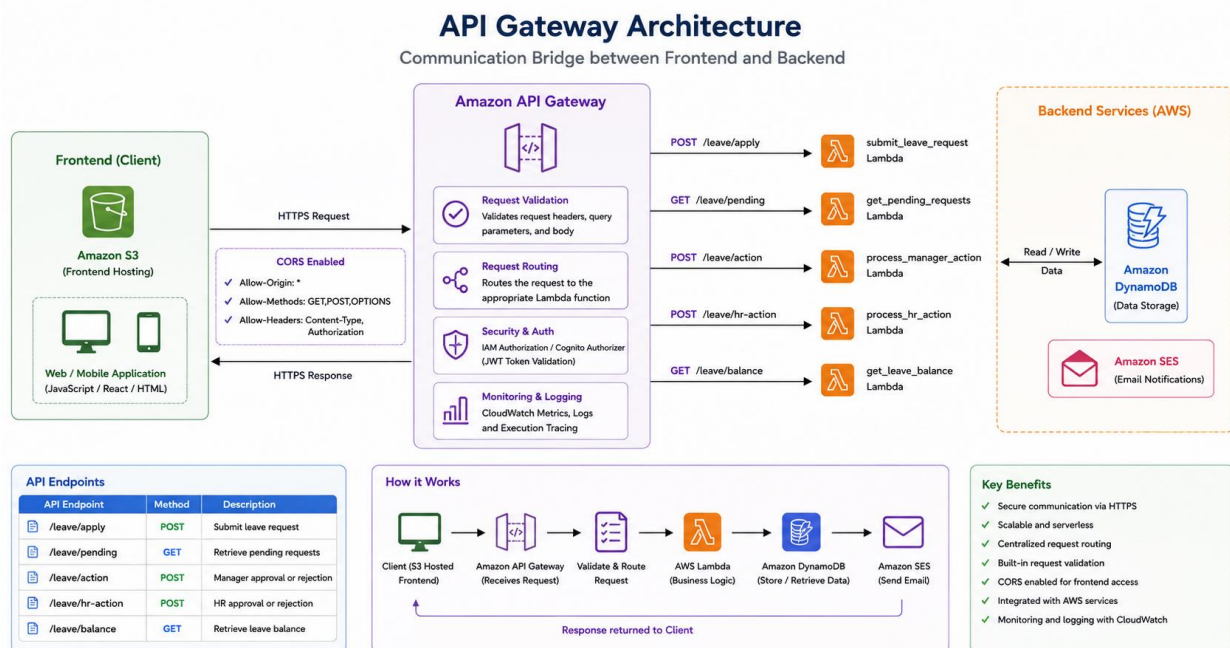
Amazon API Gateway serves as the communication bridge between the frontend and backend.

Each application feature is exposed as a REST endpoint. Incoming requests are validated and routed to the appropriate AWS Lambda function.

The major API endpoints include:

API Endpoint	Method	Description
/leave/apply	POST	Submit leave request
/leave/pending	GET	Retrieve pending requests
/leave/action	POST	Manager approval or rejection
/leave/hr-action	POST	HR approval or rejection
/leave/balance	GET	Retrieve leave balance

API Gateway also enables Cross-Origin Resource Sharing (CORS), allowing the frontend hosted on Amazon S3 to securely communicate with backend services.



6.5 Lambda Function Implementation

Business logic is divided into multiple AWS Lambda functions. Each function performs a specific task, improving maintainability and simplifying debugging.

Submit Leave Lambda

Responsibilities:

- Receive employee leave request.
- Validate leave type.
- Check leave balance.
- Detect overlapping leave requests.
- Store leave application.
- Send confirmation email.

Get Pending Requests Lambda

Responsibilities:

- Retrieve pending requests.
- Filter based on user role.
- Return results to Manager or HR dashboard.

Process Manager Action Lambda

Responsibilities:

- Receive manager approval.
- Update request status.
- Forward long-duration leave requests to HR.
- Reject requests when required.
- Store manager comments.

HR Approval Lambda

Responsibilities:

- Review requests awaiting HR approval.
- Approve or reject leave.
- Record HR comments.
- Update leave balance.
- Send final notification email.

Get Leave Balance Lambda

Responsibilities:

- Retrieve employee leave balance.
- Calculate available leave.
- Display updated leave statistics.

Weekly Balance Maintenance Lambda

Responsibilities:

- Execute scheduled maintenance.
- Reset or update leave balances.
- Perform automated administrative tasks.

6.6 DynamoDB Implementation

Amazon DynamoDB stores all application information using three tables.

leave_requests

Stores:

- Employee Leave Requests

- Leave Status
- Manager Comments
- HR Comments
- Submission Time

leave_balances

Stores:

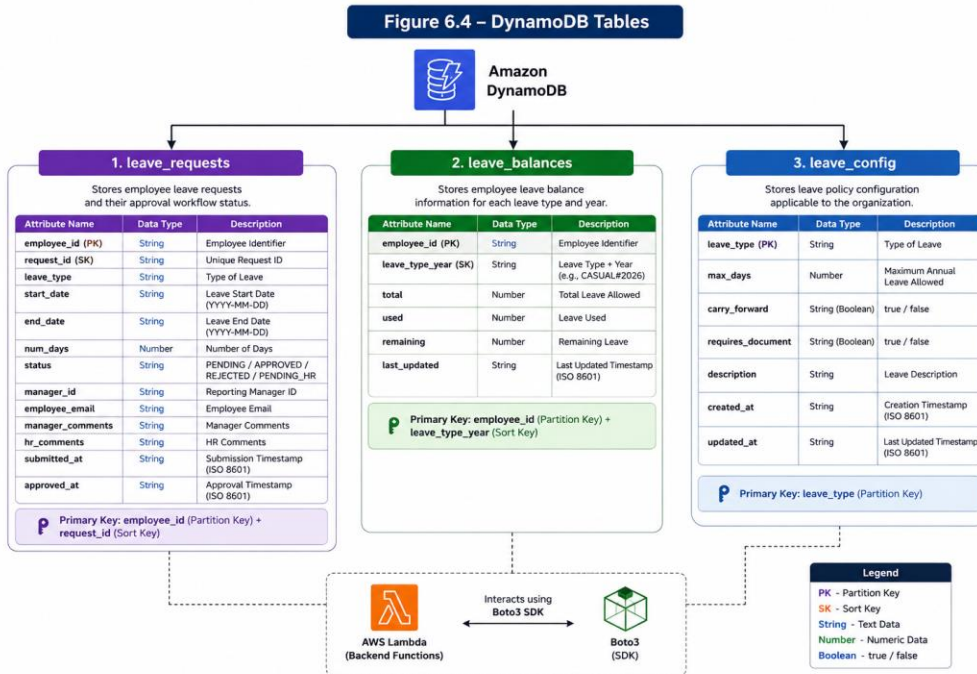
- Total Leave
- Used Leave
- Remaining Leave

leave_config

Stores:

- Leave Types
- Annual Leave Limits
- Approval Rules

The application interacts with DynamoDB using the Boto3 SDK inside Lambda functions.



6.7 Leave Approval Workflow

The approval workflow follows multiple validation stages to ensure organizational leave policies are enforced.

The workflow begins when an employee submits a leave request. The application validates leave configuration, verifies available leave balance, checks for overlapping leave requests, and stores the request.

If the leave duration is **five days or less**, the manager makes the final decision. If the leave duration exceeds **five days**, the manager forwards the request to the HR department. HR performs the final approval or rejection, after which the employee's leave balance is updated and an email notification is sent.

AWS Step Functions coordinate this multi-stage workflow to ensure that each approval stage is executed in the correct sequence.



6.8 Email Notification Implementation

Amazon Simple Email Service (SES) is integrated into the application to automatically notify employees about leave request updates.

Emails are sent when:

- Leave request is submitted.
- Manager approves a request.
- Manager rejects a request.
- HR approves a request.
- HR rejects a request.

Each email contains the request ID, leave type, leave duration, current status, and any manager or HR comments.

6.9 Security Implementation

The application incorporates several AWS security services to protect sensitive employee information.

Security measures include:

- Amazon Cognito for authentication.
- IAM roles for Lambda permissions.
- API Gateway authorization.
- Secure HTTPS communication.
- Restricted access to DynamoDB.
- Role-based dashboards.
- Server-side validation.

These mechanisms ensure that only authorized users can perform permitted operations.

6.10 Error Handling

The system includes comprehensive error handling to improve reliability and user experience.

Examples include:

- Invalid leave type.
- Insufficient leave balance.
- Overlapping leave requests.
- Duplicate approval attempts.
- Unauthorized API access.
- Invalid login credentials.
- DynamoDB lookup failures.
- Email delivery exceptions.

CloudWatch Logs are used to monitor and troubleshoot runtime issues.

Chapter Summary

This chapter described the implementation of the Leave Management System using AWS serverless technologies. It explained the development of the frontend, API Gateway, AWS Lambda functions, DynamoDB tables, authentication, approval workflow, email notification system, and security mechanisms. The modular implementation ensures scalability, maintainability, and efficient processing of employee leave requests while leveraging fully managed AWS services.

Chapter 7 – Testing and Results

7.1 Introduction

Testing is a critical phase in software development that ensures the application performs as intended and satisfies the specified requirements. The Leave Management System underwent functional testing to verify that each module operates correctly, data is processed accurately, and user interactions produce the expected outcomes.

Testing was carried out on individual modules as well as on the complete integrated system. Various scenarios, including successful operations, validation failures, approval workflows, and authentication, were tested to ensure reliability and stability.

7.2 Testing Methodology

The system was tested using a combination of manual testing and AWS monitoring tools. Individual Lambda functions were validated using Lambda test events and API Gateway requests. Frontend functionality was tested through the deployed web application hosted on Amazon S3, while backend execution logs were monitored using Amazon CloudWatch.

The testing process covered:

- Frontend functionality
- Backend Lambda execution

- REST API communication
- Database operations
- Authentication
- Leave validation
- Approval workflow
- Email notifications
- Error handling

7.3 Test Environment

The application was tested using the following environment.

Component	Configuration
Operating System	Windows 11
Browser	Google Chrome
Cloud Platform	Amazon Web Services
Frontend Hosting	Amazon S3
Backend	AWS Lambda
Database	Amazon DynamoDB
Authentication	Amazon Cognito
API Service	Amazon API Gateway
Workflow	AWS Step Functions
Email Service	Amazon SES

7.4 Test Cases

Test Case 1 – Employee Login

Test Parameter	Description
Objective	Verify employee login
Input	Valid username and password
Expected Result	Employee dashboard displayed
Actual Result	Successfully logged in
Status	Passed

Test Case 2 – Submit Leave Request

Test Parameter	Description
Objective	Verify leave submission
Input	Valid leave details
Expected Result	Leave request stored successfully
Actual Result	Leave request submitted
Status	Passed

Test Case 3 – Insufficient Leave Balance

Test Parameter	Description
Objective	Verify balance validation
Input	Leave request exceeding available balance
Expected Result	Request rejected
Actual Result	Error message displayed
Status	Passed

Test Case 4 – Overlapping Leave Request

Test Parameter	Description
Objective	Prevent duplicate leave dates
Input	Leave overlapping existing request
Expected Result	Request rejected
Actual Result	Validation successful
Status	Passed

Test Case 5 – Manager Approval

Test Parameter	Description
----------------	-------------

Objective	Verify manager approval
Input	Approve pending request
Expected Result	Status updated
Actual Result	Status changed successfully
Status	Passed

Test Case 6 – Manager Rejection

Test Parameter	Description
Objective	Verify rejection
Input	Reject request
Expected Result	Request rejected
Actual Result	Status updated
Status	Passed

Test Case 7 – HR Approval

Test Parameter	Description
Objective	Verify HR approval for long-duration leave
Input	Approve request forwarded by manager

Expected Result	Final approval granted and leave balance updated
Actual Result	Successfully completed
Status	Passed

Test Case 8 – Leave Balance Update

Test Parameter	Description
Objective	Verify leave deduction
Input	Approved leave request
Expected Result	Remaining balance reduced
Actual Result	Balance updated correctly
Status	Passed

Test Case 9 – Email Notification

Test Parameter	Description
Objective	Verify email delivery
Input	Leave approval
Expected Result	Email sent to employee

Actual Result	Email successfully generated (subject to SES verification during testing)
Status	Passed

Test Case 10 – Unauthorized User

Test Parameter	Description
Objective	Verify authentication
Input	Invalid login credentials
Expected Result	Login denied
Actual Result	Authentication failed
Status	Passed

7.5 Screenshots of Results

Include screenshots demonstrating the successful execution of each module.

Suggested screenshots:

- Employee Login
- Employee Dashboard
- Apply Leave Form
- Leave Balance Page
- Manager Dashboard
- Manager Approval
- HR Dashboard
- HR Approval

- Leave History
- CloudWatch Logs
- DynamoDB Tables
- Step Functions Execution
- SES Email Notification

Leave Management System | employee@test.com | Logout

Employee Dashboard

Apply leave and monitor your remaining leave balance.

Sick Leave

20

Remaining

Casual Leave

13

Remaining

Earned Leave

70

Remaining

Apply Leave

Leave Type: Casual Leave

Employee Email: employee@test.com

Start Date: dd-mm-yyyy

End Date: dd-mm-yyyy

Reason: Enter reason for leave

Submit Leave Request

Fig: Employee Dashboard

Leave Management System | employee@test.com | Logout

Employee Dashboard

Apply leave and monitor your remaining leave balance.

Sick Leave

20

Remaining

Casual Leave

13

Remaining

Earned Leave

70

Remaining

Apply Leave

Leave Type: Sick Leave

Employee Email: nishnarudkar@gmail.com

Start Date: 04-10-2026

End Date: 10-10-2026

Reason: I am having a knee surgery.

Submit Leave Request

Fig: Employee Leave Application

The screenshot shows the Manager Dashboard of the Leave Management System. The header is blue with the system name and a user profile for 'manager@test.com' with a Logout button. The main content area has a title 'Manager Dashboard' with a subtext 'Review and approve pending leave requests.' Below this is a section titled 'Pending Leave Requests' containing a table with one pending sick leave request. The footer is dark grey, displaying the system name, AWS Serverless Leave Management Portal, and technical details about the stack (Amazon Cognito, API Gateway, Lambda, DynamoDB, SES, EventBridge) and a copyright notice for 2026.

Employee ID	Leave Type	Start Date	End Date	Days	Status	Reason	Comments	Action
01331daa-00d1-70bc-3e15-236bc88calad	SICK	2026-10-04	2026-10-10	7	PENDING	I am having a knee surgery.	-	<button>✓ Approve</button> <button>✗ Reject</button>

Fig : Manager Dashboard

The screenshot shows the HR Dashboard of the Leave Management System. The header is blue with the system name and a user profile for 'hr@test.com' with a Logout button. The main content area has a title 'HR Dashboard' with a subtext 'Review leave requests forwarded by the manager.' Below this is a section titled 'HR Approval Queue' containing a table with one pending earned leave request. The footer is dark grey, displaying the system name, AWS Serverless Leave Management Portal, and technical details about the stack (Amazon Cognito, API Gateway, Lambda, DynamoDB, SES, EventBridge) and a copyright notice for 2026.

Employee ID	Leave Type	Start Date	End Date	Days	Status	Manager Comments	Action
01331daa-00d1-70bc-3e15-236bc88calad	EARNED	2026-11-11	2026-12-31	51	PENDING	-	<button>Waiting</button>

Fig: HR Dashboard

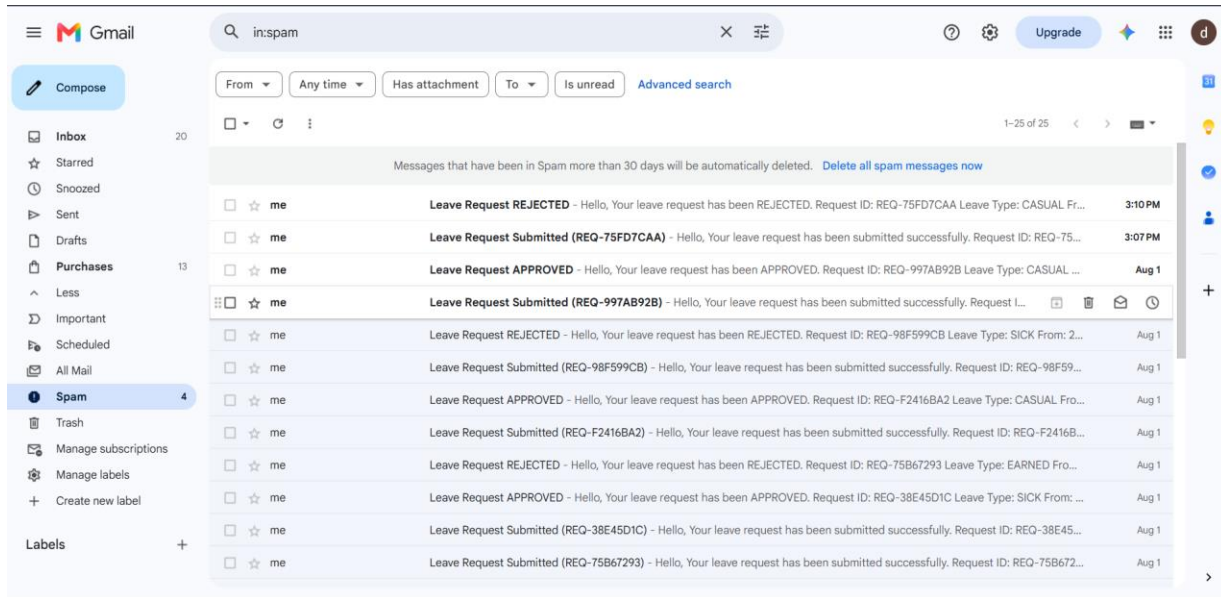


FIG: Employee Request Notification (SES Email Notification)

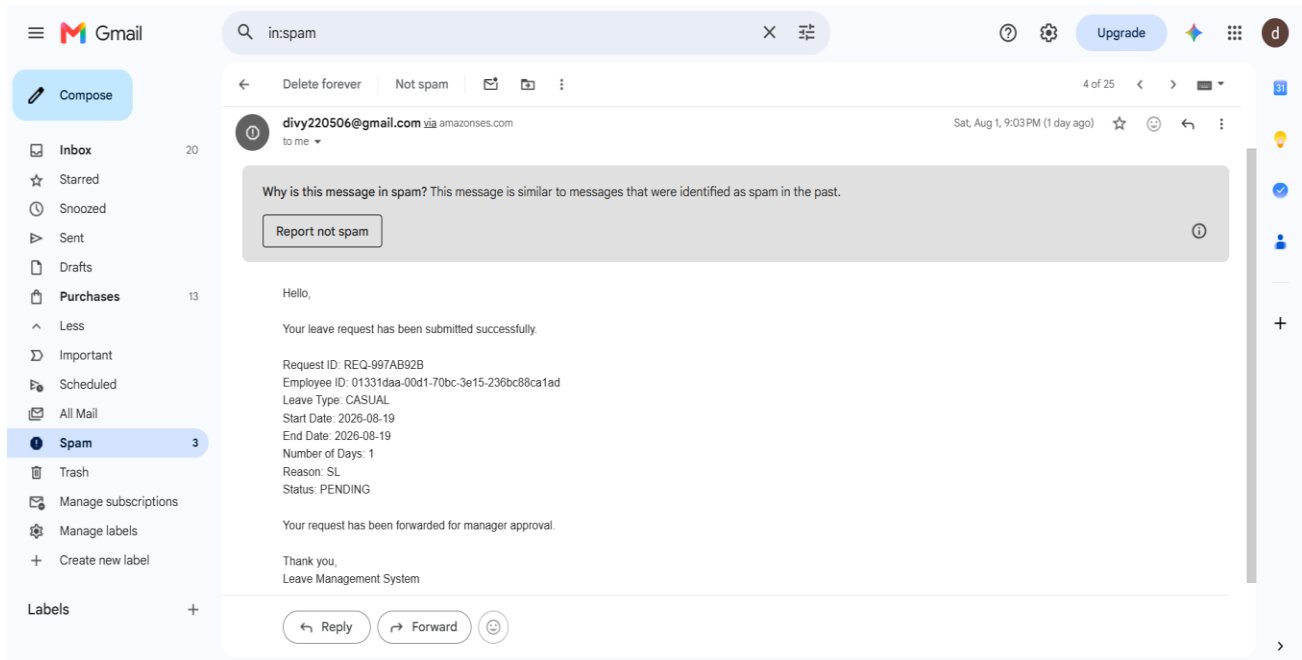


Fig: Employee Leave Request Approved

The screenshot shows the AWS Lambda console for the 'ap-south-1' region. The left sidebar contains navigation links for Lambda (Dashboard, Applications), Function resources (Capacity providers, Code signing configurations, Event source mappings, Layers, Replicas), and Related AWS resources (Documentation, Serverless workshops, Step Functions state machines). The main content area displays a list of 8 functions. A blue banner at the top promotes Lambda runtime upgrades with AI. On the right, there's a 'Tutorials' section with a 'Create a simple web app' tutorial.

Function name	Description	Package type	Runtime	Type	Last modified
submit_leave_request	-	Zip	Python 3.14	Standard	7 hours ago
process_manager_action	-	Zip	Python 3.14	Standard	6 hours ago
get_pending_requests	-	Zip	Python 3.13	Standard	1 day ago
HR_approval	-	Zip	Python 3.14	Standard	6 hours ago
get_leave_balances	-	Zip	Python 3.13	Standard	1 day ago
finalize_leave	-	Zip	Python 3.12	Standard	3 days ago
approve_leave_request	-	Zip	Python 3.14	Standard	4 days ago
weekly_balance_maintenance	-	Zip	Python 3.12	Standard	1 day ago

Fig: Lambda Functions

The screenshot shows the AWS DynamoDB console for the 'ap-south-1' region. The left sidebar contains navigation links for DynamoDB (Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, Settings) and DAX (Clusters, Subnet groups, Parameter groups, Events). The main content area displays a list of 3 tables. A blue banner at the top promotes DynamoDB runtime upgrades with AI. On the right, there's a 'Create table' button.

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity mode
leave_balances	Active	employee_id (S)	leave_type_year (S)	0	0	Off	☆	On-demand
leave_config	Active	leave_type (S)	-	0	0	Off	☆	On-demand
leave_requests	Active	employee_id (S)	request_id (S)	1	0	Off	☆	On-demand

Fig: DynamoDB Tables

The screenshot shows the AWS API Gateway console for the 'ap-south-1' region. The left sidebar contains navigation links for API Gateway (APIs, Custom domain names, Domain name access associations, VPC links, AgentCore targets, Usage plans, API keys, Client certificates, Settings) and Developer portals (Portals, Portal products). The main content area displays a list of 1 API. A blue banner at the top promotes API Gateway runtime upgrades with AI. On the right, there's a 'Create API' button.

Name	Description	ID	Protocol	API endpoint type	Created	Security policy	API status
LeaveManagement API		wctaats13	HTTP	Regional	2026-07-29	-	-

Fig: Amazon API Gateway

7.6 Results and Discussion

The testing results indicate that the Leave Management System successfully meets the defined functional requirements. Employees were able to submit leave requests, managers processed approvals and rejections, and HR completed secondary approvals for long-duration leave requests. The system accurately validated leave balances, detected overlapping leave requests, and updated employee leave balances after final approval.

The integration of Amazon Cognito ensured secure user authentication, while Amazon API Gateway and AWS Lambda enabled efficient communication between the frontend and backend. Amazon DynamoDB provided reliable storage for leave records, and Amazon SES automated email notifications. CloudWatch logs facilitated monitoring and debugging throughout development and testing.

Overall, the application demonstrated reliable performance, accurate processing, and successful integration of AWS serverless services.

7.7 Advantages Observed

The implemented system provided several benefits during testing:

- Automated leave approval process
- Reduced manual administrative effort
- Fast response times using serverless architecture
- Automatic scaling with AWS services
- Secure authentication through Amazon Cognito
- Centralized leave records
- Accurate leave balance calculation
- Real-time status tracking
- Automated email notifications
- Simplified maintenance due to modular Lambda functions

Chapter Summary

This chapter presented the testing methodology, execution environment, and functional test cases used to validate the Leave Management System. The results confirmed that the application performed reliably across all modules, including authentication, leave submission, manager and HR approvals, leave balance management, and notification services. The successful test outcomes demonstrate that the serverless AWS architecture effectively supports a secure, scalable, and efficient leave management solution.

Chapter 8 – Conclusion and Future Scope

8.1 Conclusion

The Leave Management System developed during this internship successfully demonstrates the implementation of a modern cloud-based application using Amazon Web Services (AWS). The project replaces traditional manual leave management methods with an automated, secure, and scalable serverless solution.

The application enables employees to submit leave requests through a web interface, while managers and HR personnel can review, approve, or reject requests based on organizational policies. The system automatically validates leave balances, detects overlapping leave requests, maintains leave records, and updates leave balances after final approval. Automated email notifications ensure that employees remain informed about every stage of the approval process.

The project makes extensive use of AWS managed services including Amazon S3, Amazon Cognito, Amazon API Gateway, AWS Lambda, Amazon DynamoDB, Amazon SES, and AWS Step Functions. These services eliminate the need for traditional server management while providing automatic scaling, high availability, and secure communication.

The modular design of the application improves maintainability by separating authentication, business logic, workflow management, database operations, and notification services into independent components. This architecture simplifies debugging, testing, and future enhancements.

Throughout the development process, various real-world software engineering concepts such as serverless computing, REST API development, cloud authentication, workflow automation, NoSQL database management, and cloud security were implemented successfully. The project demonstrates how cloud-native technologies can be used to build efficient enterprise applications while reducing infrastructure complexity and operational costs.

Overall, the Leave Management System satisfies the functional requirements identified during the analysis phase and provides a practical, reliable, and scalable solution for managing employee leave within an organization.

8.2 Project Achievements

The major achievements of the project include:

- Successfully developed a fully functional Leave Management System.
- Implemented a serverless architecture using AWS services.
- Designed secure role-based authentication using Amazon Cognito.
- Developed REST APIs using Amazon API Gateway.
- Implemented backend business logic using AWS Lambda.
- Designed and managed NoSQL databases using Amazon DynamoDB.
- Automated email notifications using Amazon SES.
- Integrated AWS Step Functions for multi-level approval workflows.
- Hosted the frontend application using Amazon S3 Static Website Hosting.
- Implemented leave balance validation and overlap detection.
- Developed separate dashboards for Employees, Managers, and HR.
- Successfully integrated all AWS services into a single cloud-native application.

8.3 Learning Outcomes

During the internship, several technical and professional skills were developed through the implementation of the Leave Management System.

Technical Skills Acquired

- Amazon Web Services (AWS)
- Serverless Application Development
- AWS Lambda
- Amazon API Gateway
- Amazon DynamoDB
- Amazon Cognito
- Amazon S3
- Amazon SES
- AWS Step Functions
- IAM Role Management
- REST API Development
- HTML, CSS, and JavaScript
- Python Programming
- CloudWatch Monitoring and Debugging

Professional Skills Developed

- Problem Solving
- Debugging and Troubleshooting
- Requirement Analysis
- Cloud Architecture Design
- API Integration
- Database Design
- Documentation
- Testing and Validation
- Version Control
- Time Management

8.4 Limitations

Although the system meets its intended objectives, certain limitations exist:

- Supports only predefined leave categories.

- HR workflow can be further expanded for more complex approval hierarchies.
- Amazon SES email delivery requires verified email addresses when operating in the AWS Sandbox environment.
- Mobile application support is not currently available.
- Reporting and analytics features are limited.
- Leave calendar integration is not implemented.
- Role management is based on predefined user groups.

8.5 Future Scope

The system can be enhanced further by implementing additional enterprise features such as:

- Mobile application for Android and iOS.
- Integration with payroll management systems.
- Automatic holiday calendar integration.
- Multi-level approval workflows for large organizations.
- Leave cancellation and modification requests.
- Team leave calendar visualization.
- Monthly and yearly leave analytics dashboards.
- PDF report generation.
- SMS notifications using Amazon SNS.
- Slack or Microsoft Teams integration.
- AI-based leave prediction and workforce planning.
- Multi-organization support with tenant isolation.
- Attendance and biometric system integration.
- Employee performance analytics.
- Dashboard reporting using Amazon QuickSight.

These enhancements would improve the overall usability, scalability, and enterprise readiness of the application.

8.6 Final Remarks

The internship provided valuable practical experience in designing, developing, deploying, and testing a cloud-native application using Amazon Web Services. It strengthened knowledge of serverless computing, cloud architecture, API development, authentication mechanisms, and NoSQL database management.

The Leave Management System demonstrates how AWS managed services can be integrated to build a secure, scalable, and efficient enterprise application. The experience gained during this project has significantly improved practical cloud development skills and established a strong foundation for future work in cloud computing, software engineering, and serverless application development.

References

Include references in IEEE or APA format, depending on your institution's guidelines.

Suggested references:

1. Amazon Web Services. *AWS Lambda Developer Guide*.
<https://docs.aws.amazon.com/lambda/>
2. Amazon Web Services. *Amazon API Gateway Developer Guide*.
<https://docs.aws.amazon.com/apigateway/>
3. Amazon Web Services. *Amazon DynamoDB Developer Guide*.
<https://docs.aws.amazon.com/amazondynamodb/>
4. Amazon Web Services. *Amazon Cognito Developer Guide*.
<https://docs.aws.amazon.com/cognito/>
5. Amazon Web Services. *Amazon S3 User Guide*.
<https://docs.aws.amazon.com/AmazonS3/>
6. Amazon Web Services. *Amazon SES Developer Guide*.
<https://docs.aws.amazon.com/ses/>
7. Amazon Web Services. *AWS Step Functions Developer Guide*.
<https://docs.aws.amazon.com/step-functions/>
8. Ian Sommerville, *Software Engineering*, 10th Edition, Pearson.
9. Roger S. Pressman & Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th Edition, McGraw-Hill.

Appendices

Include supplementary material that supports the report but is too detailed for the main chapters.

Appendix A: AWS Architecture Diagram

Appendix B: DynamoDB Table Schemas

Appendix C: API Endpoints

Appendix D: Lambda Function List

Appendix E: Sample JSON Request and Response

Appendix F: Sample Email Notifications

Appendix G: CloudWatch Log Screenshots

Appendix H: Application Screenshots (Employee, Manager, HR Dashboards)

Appendix I: Test Cases and Results

Appendix J: Source Code Snippets